Plan C : Integrating Public Voices into Urban Planning: An AI-Enabled Platform for Kozhikode

MAIN PROJECT REPORT

Submitted by

AMEYA ARUL (KKE22CSD008)

RIFANA SHERIN (KKE22CSD050)

SANIA K S (KKE22CSD053)

SHREYA BAIJU (KKE22CSD057)

to

the APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree

of

Bachelor of Technology

in

Computer Science and Design

Department of Computer Science and Engineering

Government Engineering College Kozhikode

Kerala

September 10, 2025

DECLARATION

We, the undersigned, hereby declare that the main project report **Plan C : Integrating Public Voices into Urban Planning: An AI-Enabled Platform for Kozhikode**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by us under the supervision of **SHABEER K P,Associate Professor**. This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact, or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources that have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma, or similar title from any other University.

Place: Kozhikode

Date: September 10, 2025

AMEYA ARUL

RIFANA SHERIN

SANIA K S

SHREYA BAIJU

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GOVERNMENT ENGINEERING COLLEGE KOZHIKODE

CERTIFICATE

This is to certify that the report entitled **"Plan C : Integrating Public Voices into Urban Planning: An AI-Enabled Platform for Kozhikode"** submitted by **AMEYA ARUL**, **RIFANA SHERIN**, **SANIA K S**, **SHREYA BAIJU** to the **APJ Abdul Kalam Technological University** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Design** is a bonafide record of the main project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other university or institute for any purpose.

Shabeer K P	**Dr. Jayasree M**	**Dr. Jayasree M**
Associate Professor	Associate Professor	Associate Professor
Department of CSE	Department of CSE	Department of CSE
(Guide)	(Coordinator)	(Head of Department)

ACKNOWLEDGEMENT

First and foremost, we would like to express our sincere gratitude and heartful indebtedness to our guide **SHABEER K P,Associate Professor**, Department of Computer Science and Engineering for his/her valuable guidance and encouragement in pursuing this project.

We are also thankful to **Dr.Jayasree M**, Head of the Department for his help and support. We also extend our hearty gratitude to project co-ordinators, **Name of Project coordinators** Government Engineering College Kozhikode for providing necessary facilities and their sincere co-operation.

Our sincere thanks is extended to all the teachers of the Department of Computer Science and Engineering and to all my friends for their help and support.

Above all, we thank God for the immense grace and blessings at all stages of the project.

AMEYA ARUL

RIFANA SHERIN

SANIA K S

SHREYA BAIJU

ABSTRACT

The proposed plan is a citizen-centric urban planning support platform that enables users to submit civic issues in Kozhikode through a Social Media-style web interface featuring posts, upvotes, comments, and image uploads. Each user is verified and authenticated via email to ensure data integrity. Submitted complaints are prioritized using a hybrid AI-human approach: an LLM suggests solutions and classifications by comparing user inputs (including images) with trained reference datasets and existing government project plans. Issues are tagged as ”planned but unsolved” or ”not in plan,” triggering actionable suggestions aligned with public policy. A progress-tracking section transparently logs the inclusion of issues in official reports and updates on action taken. The system leverages a structured database to support continuous learning and insight generation, with key consultation points directed to planning experts regarding report format, submission timelines, and decision logic validation.

Contents

List of Figures **vii**

List of Tables **viii**

Abbreviations **ix**

Notation **ix**

1 Introduction **1**

1.1 Problem Statement 1

1.2 Motivation 1

1.3 Objectives 2

1.4 Novelty of Idea 2

1.5 Social and Industrial Relevance 3

1.5.1 Social Relevance: 3

1.5.2 Industrial Relevance: 3

2 Literature Survey **4**

2.1 Influence of External Information on Large Language Models Mirrors Social Cognitive Patterns 4

2.1.1 Introduction 4

2.1.2 Methodology 4

2.1.3 Limitations 5

2.1.4 Significance 6

2.1.5 Relevance 6

2.1.6 Conclusion 6

2.2 LLM-augmented Hierarchical Reinforcement Learning for Human-like Decision-making of Autonomous Driving . . . 7
2.2.1 Introduction . . . 7
2.2.2 Methodology . . . 7
2.2.3 Limitations . . . 8
2.2.4 Significance . . . 8
2.2.5 Relevance . . . 8
2.2.6 Conclusion . . . 8
2.3 Topic Modeling and Sentiment Analysis of Arabic News Headlines for a Societal Well-Being Scoring and Monitoring System (Jannani et al., 2025) . . . 9
2.3.1 Introduction . . . 9
2.3.2 Methodology . . . 9
2.3.3 Limitations . . . 10
2.3.4 Significance . . . 10
2.3.5 Relevance . . . 11
2.3.6 Conclusion . . . 11
2.4 Artificial Intelligence Driven Predictive Analytics for Real-Time Civic Engagement and Smart Decision-Making in Future Urban Governance . . . 12
2.4.1 Introduction . . . 12
2.4.2 Methodology . . . 12
2.4.3 Limitations . . . 13
2.4.4 Significance . . . 14
2.4.5 Relevance . . . 14
2.4.6 Conclusion . . . 15
2.5 Road Pothole Detection Based on Crowdsourced Data and Extended Mask R-CNN 15
2.5.1 Introduction . . . 15
2.5.2 Methodology . . . 15
2.5.3 Experiment Design . . . 17
2.5.4 Limitations . . . 17

2.5.5 Significance 17
2.5.6 Conclusion 17
2.6 Conclusions and Gap Analysis 18
2.6.1 Conclusion 18
2.6.2 Gap Analysis 18
3 Feasibility Study and Software Requirements Specification 20
3.1 Feasibility Study 20
3.1.1 Technical Feasibility 20
3.1.2 Economic Feasibility 20
3.1.3 Legal Feasibility 21
3.1.4 Operational Feasibility 21
3.1.5 Schedule Feasibility 21
3.2 REQUIREMENTS ELICITATION 21
3.3 Software Requirements Specification 22
3.3.1 Functional Requirements 22
3.3.1.1 Registration of New Users with Email 22
3.3.1.2 Login of Existing Users with Email 23
3.3.1.3 Create a New Post 23
3.3.1.4 Comment on an Existing Post 24
3.3.1.5 Upvote an Existing Post 24
3.3.1.6 Edit Post 25
3.3.1.7 Delete Post 25
3.3.1.8 Map to Update Complaint Progress 26
3.3.1.9 Edit Account 26
3.3.1.10 Delete Account 27
3.3.1.11 Automatic Report Generation from Post Analysis by ML 27
3.3.1.12 Update Complaint Progress by AI from External Information . . 28
3.3.1.13 Non-Functional Requirements 28
3.3.2 Software Development Requirements 29

3.3.3 Technology Stack 30
3.3.4 Platforms and Tools 31
3.3.5 Hardware Requirements 31
3.3.6 Deployment Requirements 31

4 Design and Implementation **33**

4.1 Design 33
4.1.1 System Architecture 33
4.1.1.1 Modular Design 34
4.1.1.2 GUI Design 35
4.1.2 Data Flow Diagrams 39
4.1.3 Use-Case Diagrams 40
4.1.4 Database Design 40
4.1.5 ER Diagram 43
4.1.6 Sequence Diagram 45
4.2 Implementation Steps 46
4.3 Testing 48

5 Results and Discussion **49**

5.1 Results 49
5.2 Evaluation 49

6 Conclusion and Future Works **50**

References **51**

List of Figures

4.1 System Architecture 33
4.2 Usecase-Module Mapping 35
4.3 Home Page 36
4.4 View Complaints Page 37
4.5 Complaint Registration Page 37
4.6 Admin Dashboard Page 38
4.7 Map Visualization page 38
4.8 Data Flow Diagram Level 0 39
4.9 Data Flow Diagram Level 1 39
4.10 Usecase diagram 40
4.11 ER Diagram 43
4.12 Class Diagram 44
4.13 Sequence diagram 45

List of Tables

3.1 Technology Stack Used for the Urban Issue Reporting Platform 30
3.2 Platforms and Tools for Development and Deployment 31

4.1 Modular Design 34

ABBREVIATIONS

AI – Artificial Intelligence

API – Application Programming Interface

CNN – Convolutional Neural Network

ER – Entity–Relationship

GPU – Graphics Processing Unit

GUI – Graphical User Interface

IDE – Integrated Development Environment

LLM – Large Language Model

ML – Machine Learning

NLP – Natural Language Processing

RL – Reinforcement Learning

SRS – Software Requirements Specification

UI – User Interface

URL – Uniform Resource Locator

UX – User Experience

YOLOv5 – You Only Look Once version 5

NOTATION

(List in the alphabetical order)

Chapter 1

Introduction

1.1 Problem Statement

Urban planning in Kozhikode currently lacks an efficient and transparent mechanism for incorporating public voices and real-time civic issues into official decision-making processes. Citizens face difficulty in reporting concerns, tracking the status of their complaints, and ensuring that their inputs influence planning efforts. Additionally, the absence of structured feedback loops and data-driven prioritization limits the government's ability to respond effectively to community needs. There is a critical need for an AI-enabled, citizen-centric platform that can bridge the communication gap between the public and planning authorities while streamlining issue classification, solution suggestions, and progress monitoring.

1.2 Motivation

Urban planning processes often operate in silos, with limited direct engagement from the public. In cities like Kozhikode, many civic issues remain unaddressed due to the absence of an accessible, structured, and transparent complaint and feedback system. Citizens have concerns about waste management, infrastructure gaps, or road safety — but there's no unified digital channel for submission, categorization, or follow-up. This gap inspired us to create a platform where public voices can be systematically collected, analyzed using AI, and effectively integrated into city planning decisions.

1.3 Objectives

- Build a citizen-centric platform that allows users to report urban issues via a familiar, user-friendly interface.
- Ensure complaint authenticity and integrity through email verification and user authentication.
- Leverage AI (LLM and image analysis) to classify and suggest solutions for reported issues.
- Bridge the gap between public input and government planning, by tagging issues as "planned but unsolved" or "not in plan."
- Maintain transparency and accountability through a real-time progress-tracking dashboard.
- Enable continuous improvement and data learning through a structured, queryable backend.

1.4 Novelty of Idea

- Combines LLM-based natural language and image understanding to prioritize civic issues.
- Integrates social media-style UX (posts, upvotes, comments) for reporting and community engagement.
- Uses a hybrid AI-human approach to suggest actionable steps based on existing government plans.
- Includes a transparent feedback mechanism that tracks whether issues are incorporated into official reports and policies.
- Promotes data-driven urban planning using a continually growing repository of citizen inputs and administrative responses.

1.5 Social and Industrial Relevance

1.5.1 Social Relevance:

- Empowers citizens, including marginalized voices, to directly influence urban development.
- Promotes transparency, trust, and participatory governance.
- Can enhance responsiveness to pressing civic concerns (e.g., potholes, garbage accumulation, waterlogging).

1.5.2 Industrial Relevance:

- Offers municipal bodies a scalable, AI-supported tool for issue prioritization and planning validation.
- Can be integrated with existing urban data systems or e-governance platforms.
- Applicable to urban planning consultancies, civic tech startups, and smart city mission implementations across India and globally.

Chapter 2

Literature Survey

2.1 Influence of External Information on Large Language Models Mirrors Social Cognitive Patterns

2.1.1 Introduction

This paper explores the influence of external information on Large Language Models (LLMs) and their integration into society as AI agents that process and generate information. The key question is the extent to which external statements and opinions affect LLMs' "cognition" and "behaviors." Inspired by social cognitive theory, the study examines whether LLMs exhibit similar learning and influence patterns to humans. The motivation lies in the increasing use of LLMs in applications like news generation and social media, where they can shape discourse, making susceptibility to external influence an important ethical concern.

The study conceptualizes LLM interaction as a triad: textual environment (observed texts), internal memory (cognition), and generated output (behavior).

2.1.2 Methodology

Three main experiments were conducted:

Experiment 1: Memory Modification

Investigated how external objective counterfactual statements (e.g., "water itself is highly toxic") can modify LLM knowledge.

Materials: 20 counterfactual statements spanning commonsense, fake news, fictional stories, and mathematical domains, presented with different credibility levels: Twitter, blogs, news reports, and research papers.

Methods:

- Information Acquisition: In-context prompts vs. fine-tuning (LoRA and full).
- Questions: Nine questions (direct, indirect, peripheral) probed memory changes.
- Evaluation: 10 human annotators labeled correctness, with strong agreement (Fleiss' Kappa = 0.806). Automatic ChatGPT-based evaluation was also used.

Experiment 2: Opinion Formation

Explored whether external subjective opinions influenced LLM viewpoints.

Materials: 20 debate topics with opposing opinions; texts written as Twitter posts, forum posts, speeches, and research papers.

Probing: Five multiple-choice polling questions per topic (scores from -2 to +2).

Factors: Source authority, social identity (in-group, neutral, out-group), and social roles (celebrity, government).

Experiment 3: Social Media Behavioral Decisions

Examined how memory and opinion changes translate into sharing and replying behaviors.

Materials: 20 fictional events and 1600 Twitter posts with different emotions.

Measurements: Sharing willingness (-2 to +2), reply generation, sentiment analysis (VADER, LIWC), and author popularity.

LLMs Tested

ChatGPT, Alpaca, Vicuna, and Llama 3.

2.1.3 Limitations

- Avoids anthropomorphic conclusions.
- Simulated materials may not capture real-world complexity.

- Opinion surveys sensitive to prompt design.
- Limited LLM set; results may not generalize.
- Effects of alignment tuning (e.g., RLHF) require further research.

2.1.4 Significance

- Improves understanding of LLM cognition and parallels with human behavior.
- Highlights the "imitation paradox"—LLMs replicate human biases as they become more human-like.
- Informs ethical AI development and warns against misuse.
- Calls for safeguards against manipulation throughout the LLM lifecycle.
- Guides future research on mitigating authority bias, in-group bias, positivity bias, and emotional contagion.

2.1.5 Relevance

Relevant to AI ethics, human-AI interaction, computational social science, policy-making, and LLM development.

2.1.6 Conclusion

The study shows that LLMs are strongly influenced by external information, exhibiting authority bias, in-group bias, positivity bias, and emotional contagion. While these patterns make them human-like, they also create vulnerabilities. Larger model sizes and alignment tuning may amplify these effects. The paper calls for robust measures to detect and mitigate such influences.

2.2 LLM-augmented Hierarchical Reinforcement Learning for Human-like Decision-making of Autonomous Driving

2.2.1 Introduction

This paper focuses on the challenge of generalization in autonomous driving RL systems, particularly in rare "long-tail" scenarios (e.g., emergency vehicles, unexpected pedestrian intrusions). Traditional RL systems lack contextual reasoning and common-sense knowledge. LLMs, with their strong reasoning abilities, offer a solution by acting as high-level planners, guiding RL agents' low-level control—similar to human brain-cerebellum cooperation. The integration faces two challenges: (1) temporal mismatch between long-term LLM goals and short-term RL actions, and (2) reward design aligning human intent with driving goals.

2.2.2 Methodology

High-Level Decision (LLM-based)

- Converts raw sensor data into natural language.
- Generates a long-term goal, a meta-action (e.g., SLOW, FAST, LEFT), and an explanation.
- Uses Chain-of-Thought prompting and operates at a reduced frequency.

Low-Level Planning (RL-based)

- Receives LLM goals and environment states.
- Learns short-term actions (steering, acceleration) in a goal-conditional MDP.
- State representations include ego vehicle motion, surrounding vehicles, and visual features.
- Reward function: r_{goal}, r_{vel}, r_{smooth}, r_{safety}.
- Goal Gradient-based Transfer (GGT) accelerates learning by embedding gradients toward goals.

Human-in-the-Loop Reward Design

- LLM generates diverse reward functions.
- RL agents are pre-trained, and their trajectories evaluated by human drivers/passengers.
- Preferences are ranked using Elo ratings; the best reward function is iteratively refined.

Simulation Platform and Metrics

Experiments are run in CARLA. Metrics include success rate, collision rate, travel duration, average speed, and comfort.

2.2.3 Limitations

- Potential unsafe or hallucinated goals from LLM.
- Currently simulation-only; real-world validation needed.
- Scalability of human feedback can be challenging.

2.2.4 Significance

- Bridges the gap between RL and LLM reasoning.
- Enables human-like decision-making using common sense.
- Improves generalization, interpretability, and safety.
- Introduces a novel reward design pipeline using human preferences.
- Speeds up policy learning with GGT.

2.2.5 Relevance

Relevant to autonomous driving, RL research, human-AI collaboration, AI safety, and robotics.

2.2.6 Conclusion

The framework effectively combines LLM reasoning with RL control, significantly improving safety, robustness, and human-likeness in autonomous driving. Though limited to simulations and prone to LLM hallucinations, this approach demonstrates the strong potential of integrating LLMs into hierarchical RL systems.

2.3 Topic Modeling and Sentiment Analysis of Arabic News Headlines for a Societal Well-Being Scoring and Monitoring System (Jannani et al., 2025)

2.3.1 Introduction

This paper presents a robust NLP-based framework to analyze Arabic news headlines for real-time societal well-being monitoring. Using a large dataset—over 418,000 headlines from the Moroccan News Arabic Dataset (MNAD)—the study focuses on extracting thematic trends and public sentiment across key areas such as politics, health, and education. Rather than treating news as static information, the framework repurposes it into a dynamic tool for societal insight.

The work is grounded in the broader vision of turning digital media into meaningful social analytics. By integrating deep learning and sentiment scoring, the system aims to provide continuous, data-driven assessments of public mood and life quality across Arabic-speaking regions, offering a scalable foundation for regional analysis and governance.

2.3.2 Methodology

The proposed framework integrates several components:

Topic Modeling with LDA

- Latent Dirichlet Allocation (LDA) is used to discover hidden topics (e.g., environment, economy) in an unsupervised way, structuring the raw news dataset into semantic categories.

Word Embeddings with AraBERT and ASAFAYA

- Pre-trained Arabic language models are used to create rich vector representations of text, improving contextual understanding for downstream tasks.

Classification Models

- Traditional ML methods: SVM, Random Forest, Logistic Regression.

- Deep learning methods: GRU, Bi-GRU, LSTM.
- The GRU combined with ASAFAYA embeddings achieves the highest classification accuracy of 94.74%.

Sentiment Scoring System

- A composite well-being score is generated by aggregating sentiment values from different thematic categories, normalized to reflect daily, monthly, and quarterly trends.

Societal Monitoring Dashboard

- Combines topic distribution (objective indicators) and sentiment scores (subjective indicators) for an overall well-being index.

2.3.3 Limitations

- **Lack of Deployment:** The system is not implemented in a live, real-time setting; results are based on static data.
- **Limited Transparency in Sentiment Analysis:** The exact tools or models used for sentiment scoring are not elaborated.
- **Generalization Risk:** While Modern Standard Arabic (MSA) supports broader scalability, the headlines may still carry region-specific biases or cultural interpretations.
- **Model Interpretability:** Deep learning models like GRU lack explainability, limiting trust in decision-making.
- **Temporal Adaptability:** Topic and sentiment dynamics over time (e.g., evolving political language) are not explicitly modeled.

2.3.4 Significance

- **Validates NLP for Social Monitoring:** Demonstrates the use of NLP and sentiment analysis for large-scale social monitoring.

- **News as Societal Cues:** Establishes that news headlines reflect emotional and societal cues, not just factual reporting.
- **Language-Specific Models:** Emphasizes the importance of language-specific models such as AraBERT and ASAFAYA, supporting the case for localization in your platform.
- **Hybrid Indicators:** Proposes a hybrid indicator model (subjective + objective), reinforcing the credibility of well-being analytics in civic tech.
- **High Accuracy:** The high accuracy and layered framework suggest strong viability for use in government dashboards or public-facing monitoring tools.

2.3.5 Relevance

This paper is highly relevant to the Plan C project and directly supports its goals:

- **NLP on User Feedback:** Reinforces your approach to analyze user-generated text (citizen feedback, reports) using NLP.
- **Topic Modeling and Sentiment:** Justifies the inclusion of topic modeling and sentiment scoring in your prioritization pipeline.
- **Dashboard Design:** Suggests a practical format for your real-time dashboards, integrating both objective indicators (e.g., issue type) and subjective ones (e.g., urgency/emotion).
- **Localization:** Highlights the need for Arabic-specific models, echoing your emphasis on localization and inclusivity.
- **Scoring and Visualization:** Offers a scoring and visualization strategy that aligns with your project's goal of creating transparent, actionable civic engagement tools.

2.3.6 Conclusion

The study introduces a data-driven, Arabic NLP-based societal monitoring framework. By combining topic modeling, sentiment analysis, and deep learning, it creates an integrated

index of public sentiment and life quality. Though currently limited to static datasets, the framework provides a strong foundation for real-time, multilingual monitoring systems that can inform decision-makers and enhance citizen participation.

2.4 Artificial Intelligence Driven Predictive Analytics for Real-Time Civic Engagement and Smart Decision-Making in Future Urban Governance

2.4.1 Introduction

This paper explores the role of Artificial Intelligence (AI), specifically Machine Learning (ML), in enhancing civic engagement and real-time urban decision-making. The growing urban complexity and limitations of bureaucratic responsiveness have led city administrators to rely on AI-based predictive tools. The paper identifies a major limitation in traditional smart city frameworks: their retrospective nature and lack of contextual sensitivity, which limits their responsiveness to dynamic urban conditions and citizen feedback.

To address this, the study proposes an AI-enabled governance framework that leverages real-time feedback from citizens, IoT sensors, and predictive ML models. This framework aims to increase transparency, responsiveness, and inclusivity in urban management—ensuring that city governance is not only data-informed but citizen-centric and ethically grounded.

2.4.2 Methodology

The proposed system integrates multiple components that work synergistically to enable proactive governance:

Data Ingestion and Feedback Loop

- Real-time civic data is collected from citizens and IoT-based sensors.
- Feedback is categorized and prioritized through sentiment analysis and issue tagging.

Machine Learning Core

- Predictive analytics identify trends and potential problem areas.
- Anomaly detection and classification models help highlight urgent or emergent issues.

AI-Guided Decision Engine

- Combines current issue trends with historical data to suggest optimal responses.
- Leverages citizen sentiment and contextual metadata for weighted prioritization.

Ethical and Inclusive AI Design

- Emphasizes fairness, equity, and data privacy in all stages—from data collection to model inference.

While detailed algorithmic specifications (e.g., model architecture or training data) are not provided, the system is designed to adapt in real time using contextual awareness and human feedback, allowing administrators to make informed, timely decisions.

2.4.3 Limitations

- **Lack of Specific ML Models:** The paper does not detail the exact machine learning algorithms used for prediction or prioritization, which limits reproducibility and technical assessment.
- **Conceptual Framework:** The research remains at a conceptual level without empirical validation or system deployment, relying on hypothetical examples.
- **Ambiguity in Evaluation Metrics:** No quantitative results or model benchmarks are presented to assess effectiveness, accuracy, or responsiveness.
- **Scalability and Infrastructure Needs:** The paper assumes the existence of sufficient digital infrastructure (e.g., real-time IoT data pipelines), which may not be feasible in all urban contexts, especially in the Global South.

2.4.4 Significance

- **Advances AI-Governance Integration:** Bridges AI and urban governance, proposing a paradigm shift toward proactive, data-driven city management.
- **Citizen-Centric AI:** Reinforces the idea that AI should serve citizen needs rather than only administrative efficiency—positioning civic participation as central to smart governance.
- **Strategic Planning:** The framework allows policymakers to anticipate issues rather than merely respond, supporting more strategic and sustainable urban development.
- **Ethical AI Awareness:** Explicitly incorporates privacy, fairness, and inclusivity, which are often overlooked in AI applications in governance.

2.4.5 Relevance

This paper is highly relevant to the Plan C project and directly supports its goals in several ways:

- **Problem Prioritization:** Reinforces your platform's goal of identifying and ranking civic issues using real-time feedback, aligning with your ML-based issue classification and urgency modeling.
- **Data-Driven Decision Support:** Justifies integrating a decision layer that recommends actions or escalations based on predictive insights—an essential feature in your public participation dashboard.
- **Ethical AI Alignment:** Emphasizes inclusive AI development and bias mitigation, echoing your project's commitment to transparency and fair civic representation.
- **Urban Governance Integration:** Supports the ambition to present your project to government stakeholders as a policy-relevant digital tool for responsive planning.

2.4.6 Conclusion

The study lays the groundwork for AI-enhanced, citizen-focused urban governance by presenting a predictive analytics framework that combines ML, IoT, and feedback systems. While theoretical, it emphasizes key principles vital for modern city management—real-time responsiveness, ethical AI use, and active citizen inclusion. For your project, this paper serves as a foundational reference supporting the platform's conceptual design and strategic vision. It also highlights future paths for system scaling and deeper ML integration, which your technical implementation can help fulfill.

2.5 Road Pothole Detection Based on Crowdsourced Data and Extended Mask R-CNN

2.5.1 Introduction

This paper addresses the problem of detecting road potholes automatically to improve road safety and maintenance. Manual pothole inspections are slow, expensive, and hazardous. To address these issues, the study proposes a deep learning-based solution that detects potholes, identifies their exact shapes, and estimates their sizes from images.

The proposed work focuses on:

- Building a dataset of 1450 diverse road images (dry, wet, different lighting and surfaces).
- Extending Mask R-CNN by replacing its ResNet backbone with a ConvNeXt-T backbone for better feature extraction.
- Using the model to detect potholes at pixel level and calculate their size with high accuracy.

2.5.2 Methodology

The detection framework involves several stages:

Dataset Preparation

- Road images collected from Kaggle, Google, and crowdsourced contributions.
- Images cover varied weather, lighting, and surface conditions.
- Dataset split into 70% training, 20% validation, 10% testing.

Feature Extraction with ConvNeXt-T

- ConvNeXt-T replaces ResNet as the backbone.
- Uses depthwise 7x7 convolutions, GELU activation, LayerNorm, and a 3:3:9:3 block ratio.
- This backbone improves edge and texture extraction for small and irregular potholes.

Region Proposal Network

- Generates candidate regions (anchors of sizes 64, 128, 256, 512 with different aspect ratios).
- Filters overlapping proposals using Non-Maximum Suppression.

ROI Align Layer

- Precisely crops proposed regions without misalignment using bilinear interpolation.

Detection and Segmentation

- The model has three output branches:
    1. Classification to confirm the presence of a pothole.
    2. Bounding box regression to refine location and size.
    3. Mask segmentation using a Fully Convolutional Network to get the exact pixel-level shape.

2.5.3 Experiment Design

- Environment: NVIDIA Tesla T4 GPU, PyTorch.
- Training setup: batch size 2, AdamW optimizer, learning rate 0.001 with warm-up and step decay.
- Evaluation metrics: Precision, Recall, mAP@0.5, mAP@0.75, APS, APM, APL.

2.5.4 Limitations

- Limited dataset size; more global images are needed.
- Very small potholes remain hard to detect due to feature loss during downsampling.
- Wet or shadowed surfaces can confuse the segmentation, leading to minor errors.

2.5.5 Significance

- Achieves a mAP@0.5 of 92.1 on the custom dataset, higher than ResNet and Swin Transformer backbones.
- Reduces missed detections, false positives, and duplicate detections.
- Estimates pothole area with only 3.19% error, enabling precise size measurements for repairs.
- Ablation studies show Smooth L1 loss significantly improves small pothole detection.

2.5.6 Conclusion

This research demonstrates that combining Mask R-CNN with a ConvNeXt-T backbone can accurately detect, segment, and measure road potholes. The approach improves feature extraction over older backbones and provides reliable results across different road conditions, enabling automation in road maintenance and distress analysis.

2.6 Conclusions and Gap Analysis

2.6.1 Conclusion

The reviewed literature spans a range of advanced AI and machine learning applications in domains such as autonomous decision-making, sentiment-aware urban planning, real-time civic issue detection, and language model behavior. Key technological trends include the integration of deep neural networks, transformer-based NLP models, and large language models (LLMs) into practical, data-driven frameworks for improving public infrastructure, citizen engagement, and urban management.

Several works demonstrate the potential of LLMs to enhance decision-making (e.g., in autonomous driving or policy alignment), while others leverage CNN-based image processing and topic modeling for urban monitoring and sentiment analysis. These studies collectively emphasize scalability, interpretability, and responsiveness as key attributes in AI-driven civic systems. The inclusion of real-time feedback loops, multilingual support, and diverse data sources highlights the growing maturity of AI solutions in addressing complex societal challenges.

2.6.2 Gap Analysis

Despite the progress observed in the existing literature, several key gaps remain that provide opportunities for future work:

- **Integration Gap:** Most studies focus on either image-based detection or text-based sentiment analysis. Few integrate both modalities into a unified framework for holistic urban issue reporting and prioritization.

- **Citizen-Centric Decision Feedback:** While many systems collect user data, fewer provide real-time visibility into issue resolution or allow bidirectional communication between citizens and officials.

- **Localized and Scalable Implementation:** Many works rely on specific datasets or simulation environments (e.g., CARLA, Arabic news corpora), limiting generalizability

to new cities, languages, or infrastructure contexts without additional customization.

- **Efficient Labeling Strategies for Urban Datasets:** Deep learning models for civic image or text classification require extensive labeled data. However, scalable labeling strategies like active learning, weak supervision, or gamified crowd-sourcing remain underutilized in smart city contexts and could significantly accelerate dataset growth.

- **Dynamic Prioritization Mechanisms:** While issue detection is well-studied, mechanisms for dynamically prioritizing civic issues based on votes, urgency, and spatial distribution are still in early stages.

Chapter 3

Feasibility Study and Software Requirements Specification

3.1 Feasibility Study

This chapter evaluates the viability of the proposed platform for citizen-centric urban planning through multiple lenses—technical, economic, time, legal, and operational. Each aspect is analyzed below to determine whether the project can be realistically implemented and sustained.

3.1.1 Technical Feasibility

The platform will use a React.js frontend and Django backend to support user authentication, image uploads, voting, and report generation. Complaint classification will be performed using pre-trained CNN models (e.g., MobileNet) for images and transformer-based NLP models (e.g., BERT) for text. A PostgreSQL database will manage structured data. All technologies are widely used and compatible, making the system technically feasible with moderate development effort.

3.1.2 Economic Feasibility

Initial development can leverage open-source tools and free-tier cloud services (e.g., Firebase, Vercel). Key costs involve domain, hosting, and optional API usage for ML models. Operational costs are minimal for a prototype and scalable for wider deployment. The project

aligns well with smart city funding opportunities, ensuring economic sustainability.

3.1.3 Legal Feasibility

The platform will follow India's Digital Personal Data Protection Act for handling location and image data. Email-based authentication ensures accountability. As AI only assists in classification, decision-making remains human-supervised, maintaining legal compliance. Accessibility and transparency standards will be upheld per public governance guidelines.

3.1.4 Operational Feasibility

The platform's social media-style interface ensures usability, even for digitally inexperienced citizens. Its modular design allows for easy scaling and integration with city workflows. Success depends on training municipal staff to interpret reports and act on them, but the system is operationally viable with proper stakeholder engagement.

3.1.5 Schedule Feasibility

The project can be developed in 8–12 weeks. Core features like issue submission, voting, and ML-based tagging can be implemented in Phase 1, followed by progress tracking and report automation. With a small, dedicated team and existing frameworks, timelines are realistic for academic or pilot-level deployment.

3.2 REQUIREMENTS ELICITATION

Stakeholder Identification

- **Users** – Citizens who submit complaints regarding civic issues and track their resolution status through the system.
- **System Developers** – The project team responsible for designing, developing, and maintaining the platform to ensure functionality and reliability.
- **Government Officials** – Town planning authorities and related departments who review complaints, assign priorities, and update resolution status.

- **Academic Supervisors** – Faculty members guiding and monitoring the project, ensuring adherence to academic and technical standards.

Elicitation Techniques

- **Interviews** – Conducted at the Town Planning Office with the Town Planner to understand workflows, pain points, and requirements from the officials' perspective.
- **Observations** – Monitoring existing infrastructure, living standards, and civic conditions to identify recurring issues and real-world constraints.
- **Document Analysis** – Reviewing town planning records, official reports, and past project documents to understand historical problems and approaches taken.
- **Brainstorming** – Internal team discussions that led to innovative features such as image-based complaint submission, report generation, and an integrated platform to bridge the gap between officials and citizens.

3.3 Software Requirements Specification

Urban areas like Kozhikode often lack a proper system for citizens to report civic issues and track their resolution. This leads to communication gaps between the public and planning authorities.

This Software Requirements Specification (SRS) outlines an AI-powered platform that enables users to report urban problems, which are then classified and addressed using AI tools like language models and image analysis. The system also includes user authentication, real-time tracking, and structured data storage to support better urban planning decisions.

3.3.1 Functional Requirements

3.3.1.1 Registration of New Users with Email

Use case name: Registration of new users in the app with email

Objective: To include the user in the system and verify the email address.

Precondition: A device with an internet connection.

Postcondition: User registered and assigned a unique customer ID.

Flow of Events (Basic Flow):

1. The user opens the application.
2. User registers by providing details including their email address.
3. A verification link/code is sent to the email.
4. User verifies by clicking the link or entering the code.

3.3.1.2 Login of Existing Users with Email

Use case name: Entry of user into the web app after signup

Objective: To authenticate a user by logging in.

Precondition: Internet connection and registered account.

Postcondition: User can access their dashboard.

Flow of Events (Basic Flow):

1. The user opens the application.
2. User enters registered email and password.
3. System authenticates credentials.
4. On success, the user is redirected to their dashboard.

3.3.1.3 Create a New Post

Use case name: Create a new urban issue post

Objective: To allow users to report civic issues.

Precondition: User is logged in.

Postcondition: A new post is created and visible.

Flow of Events:

1. User navigates to "Create New Post".
2. Provides description and uploads image (optional).
3. Provides location (map pin or address).
4. Submits the post.
5. System processes post with AI classification and prioritization.

3.3.1.4 Comment on an Existing Post

Use case name: Add a comment to an existing post
Objective: Provide additional context or discussion.
Precondition: Logged in and viewing a post.
Postcondition: Comment is added to the post.

Flow of Events:

1. User views a post.
2. Enters a comment in the field.
3. Submits the comment.
4. Comment is displayed under the post with attribution.

3.3.1.5 Upvote an Existing Post

Use case name: Upvote a post
Objective: Allow users to show support.
Precondition: Logged in and viewing a post.

Postcondition: Upvote count is increased.

Flow of Events:

1. User views a post.
2. Clicks "Upvote".
3. System increments upvote count and records user's upvote.

3.3.1.6 Edit Post

Use case name: Edit an existing post
Objective: Modify content of own post.
Precondition: Logged in as post owner.
Postcondition: Post content updated.

Flow of Events:

1. User navigates to their post.
2. Selects "Edit Post".
3. Updates content (description, image, location).
4. Saves changes.
5. System updates the post.

3.3.1.7 Delete Post

Use case name: Delete an existing post
Objective: Remove own post.
Precondition: Logged in as post owner.
Postcondition: Post is removed.

Flow of Events:

1. User navigates to their post.
2. Selects "Delete Post".
3. Confirms deletion.
4. System removes post and associated data.

3.3.1.8 Map to Update Complaint Progress

Use case name: Visualize and update complaint progress on a map
Objective: Show issue status and allow authorized updates.
Precondition: Logged in, location data available.
Postcondition: Map updated in real-time.

Flow of Events:

1. User accesses the map interface.
2. Map displays color-coded markers for each issue.
3. User clicks marker to view details and current status.
4. AI updates the status (reported → in-progress → complete).
5. Map marker color and post status update in real-time.

3.3.1.9 Edit Account

Use case name: Modify user account details
Objective: Update personal information.
Precondition: User is logged in.
Postcondition: Account details updated.

Flow of Events:

1. User opens Account Settings.

2. Modifies fields.

3. Saves changes.

4. System updates account information.

3.3.1.10 Delete Account

Use case name: Permanently delete user account

Objective: Remove account and all data.

Precondition: Logged in and chooses to delete account.

Postcondition: Account and data removed.

Flow of Events:

1. User opens Account Settings.

2. Selects "Delete Account".

3. Confirms action.

4. System permanently removes account and data.

3.3.1.11 Automatic Report Generation from Post Analysis by ML

Use case name: Generate automatic reports from ML analysis

Objective: Provide summarized data and insights for planners.

Precondition: Sufficient data available; ML models operational.

Postcondition: Reports generated and available.

Flow of Events:

1. System initiates report generation periodically or on trigger.

2. ML analyzes posts (descriptions, images, upvotes, comments).

3. System compiles structured reports (PDF/CSV/dashboard).

4. Reports include trends, priorities, and recommendations.

3.3.1.12 Update Complaint Progress by AI from External Information

Use case name: Automatically update complaint progress using external data

Objective: Use external sources to keep status updated.

Precondition: External data sources are configured.

Postcondition: Complaint status updated automatically.

Flow of Events:

1. AI monitors external data sources.
2. Matches data with existing complaints.
3. Infers possible status updates.
4. Validates update using confidence thresholds.
5. Updates complaint status and logs the source.
6. Sends notification to relevant users.

3.3.1.13 Non-Functional Requirements

- **Scalability:** The system should support increasing users and posts without performance drops.
- **Performance:** Responses to user actions (e.g., submitting an issue or upvoting) should occur within 2 seconds.
- **Availability:** The system should maintain 99% uptime for critical services.
- **Security:** All user data (including images and location) must be securely stored and access controlled.
- **Usability:** Interface should be simple, mobile-friendly, and usable by citizens with basic digital literacy.

- **Maintainability:** Modular codebase should allow easy future updates.

3.3.2 Software Development Requirements

- **Frontend Framework:** React.js (or Next.js for SSR and routing).
- **Backend Framework:** Django (Python) or Node.js (Express).
- **AI/ML Integration:**
  - **Image Processing:** Use CNN-based models (e.g., MobileNet, YOLOv5) via TensorFlow or PyTorch to classify images of civic issues (e.g., garbage, potholes).
  - **Natural Language Processing (NLP):** Apply transformer-based NLP models (e.g., BERT) to analyze and tag complaint descriptions.
  - **LLM-Based Decision Support:** Use Large Language Models (LLMs) such as GPT-4, Mistral, or LLaMA to assist in contextual classification, prioritization of issues, and matching complaints with city planning documents.
  - **Complaint Classification and Ranking:** Use ML techniques such as Decision Trees, Support Vector Machines (SVMs), Artificial Neural Networks (ANNs), and ensemble models (e.g., Random Forests, Gradient Boosting) to sort, group, and rank citizen complaints and suggestions.
- **Database:** PostgreSQL for structured data, Firebase or AWS S3 for media storage.
- **APIs:** Google Maps API for geolocation; OpenAI or custom NLP APIs for LLM integration.

3.3.3 Technology Stack

Layer	Technology Used	Purpose
Frontend	React.js or Next.js	UI, routing, and client-side logic
Backend	Django (Python) or Node.js (Express)	Server-side logic, API endpoints, authentication
Database	PostgreSQL	Structured storage for users, complaints, metadata
Media Storage	Firebase Storage or AWS S3	Store uploaded images and media
AI/ML Framework	TensorFlow or PyTorch	Model development for image and text classification
Image Classification	MobileNet, YOLOv5 (CNN)	Identify urban issues in uploaded images
Text Processing (NLP)	BERT or similar transformer models	Tag and classify complaint descriptions
LLM Decision Support	GPT-4, Mistral, LLaMA	Prioritization and contextual classification of complaints
APIs	Google Maps API, OpenAI API	Geolocation and LLM inference
Authentication	Firebase Auth, JWT	Secure user login and access control
Version Control	Git + GitHub	Code management and collaboration

Table 3.1: Technology Stack Used for the Urban Issue Reporting Platform

3.3.4 Platforms and Tools

Category	Platform / Tool	Purpose
Hosting (Frontend & Backend)	Firebase, Vercel, or AWS	Deploy frontend/backend services
ML Model Hosting	Hugging Face Spaces, Replicate, or local server	Host AI models for inference
Containerization	Docker	Package and run services uniformly
CI/CD	GitHub Actions	Automated testing and deployment
Development IDE	VS Code / PyCharm	Local development
Collaboration	GitHub Projects / Notion	Task tracking and planning

Table 3.2: Platforms and Tools for Development and Deployment

3.3.5 Hardware Requirements

- **Development Machines:** At least 8 GB RAM, i5 processor, 256 GB SSD (for local development).
- **Cloud Hosting Server (Deployment):** Minimum 2-core CPU, 4 GB RAM, SSD storage.
- **AI Compute:** Optional GPU usage for training/classification, via Google Colab or cloud platforms.

3.3.6 Deployment Requirements

- **Networks / Server Infrastructure:** The platform will be deployed using cloud services such as Firebase, Vercel, or AWS to host frontend and backend services, ensuring scalability, reliability, and continuous availability.
- **Database Management:** PostgreSQL will be deployed on a secure cloud environment for structured data, with Firebase or AWS S3 handling media file storage. Regular backups and access controls will be implemented for data safety.
- **Mobile Application Development:** A responsive web interface will be optimized for mobile access. Future native app development using Flutter or React Native may be considered for wider adoption and offline support.

- **Testing and Quality Assurance:** Continuous Integration (CI) using GitHub Actions will automate testing workflows. Manual and automated tests will ensure functional correctness, performance, and usability before deployment.
- **Documentation and User Support:** Comprehensive developer and user documentation will be provided. Help sections, FAQs, and onboarding guides will support users. A feedback mechanism will be included to gather user issues and suggestions.
- **Containerization:** Docker will be used optionally to package the application for consistent deployment across environments.

Chapter 4

Design and Implementation

4.1 Design

4.1.1 System Architecture

The architecture of the Plan C platform is organized into multiple design components to clearly represent the flow of data and interaction between users, databases, and AI modules.

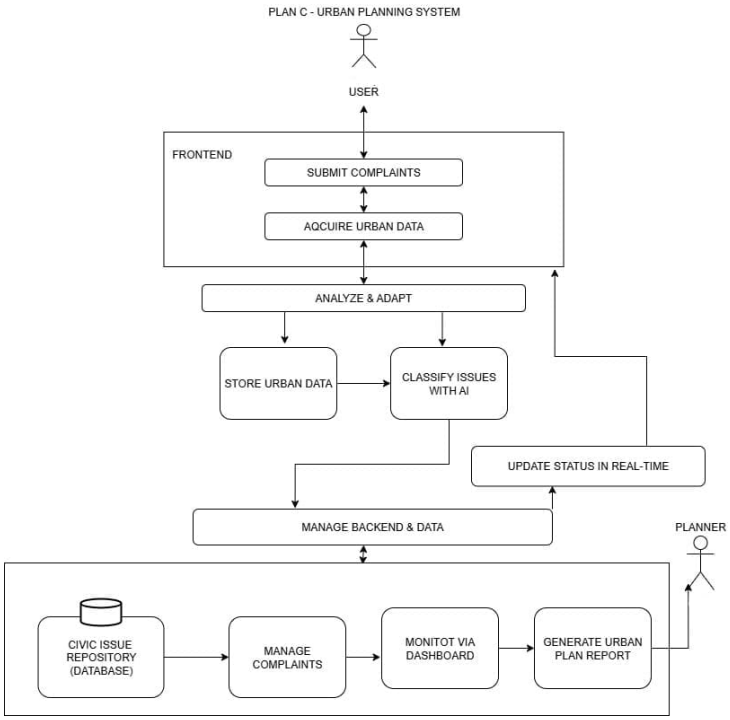

Figure 4.1: System Architecture

4.1.1.1 Modular Design

Sl. No.	Module	Inputs	Action / Process	Outputs
1	**User Authentication Module**	User credentials (username, password)	Validates credentials against the local database to ensure only registered users can access the system.	Access token / login success or failure
2	**Complaint Submission Module**	Text description, images, location data using map	Allows users to submit complaints. Data is securely stored in the cloud while references are maintained in the local database.	Stored complaint in cloud database and reference in local database
3	**Complaint Browsing and Interaction Module**	Existing complaints from database, user interactions (votes, comments)	Displays complaints in a feed interface, enables upvotes and comments linked to original complaints.	Updated complaint records with votes and comments
4	**AI-Powered Analysis Module**	Complaint data (text + images), user votes	Uses CNN for images, BERT for text, and LLM for assigning priority tags by combining AI outputs with feedback.	Classified complaints with priority tags
5	**Report Generation Module**	Analyzed complaints with classification and priority scores	Generates automatic reports and send to officials.	Visual reports and dashboard view
6	**Status Update Module**	Planner inputs (status updates)	Officials update complaint progress as not solved, in progress, or solved for transparency.	Updated complaint statuses visible to citizens on map

Table 4.1: Modular Design

Use Case (Functional Requirement)	MODULES					
	User Authentication Module	Complaint Submission Module	Complaint Browsing and Interaction Module	AI-Powered Analysis Module	Report Generation Module	Status Update Module
User Registration & Authentication	X					
Create, Edit, and Delete Posts		X				
View, Upvote, and Comment on Posts			X			
Map to Update Complaint Progress						X
Account Editing & Deletion	X					
Automatic Report Generation from Post Analysis by ML				X	X	
Update Complaint Progress by AI from External Information				X		X

Figure 4.2: Usecase-Module Mapping

4.1.1.2 GUI Design

The graphical user interface (GUI) provides citizens with options to register, log in, submit complaints, and view complaint statuses. Once submitted, complaints are displayed in a feed-like interface, where other users can upvote or comment. A map-based view highlights the geographic locations of issues and their current status.

Figure 4.3: Home Page

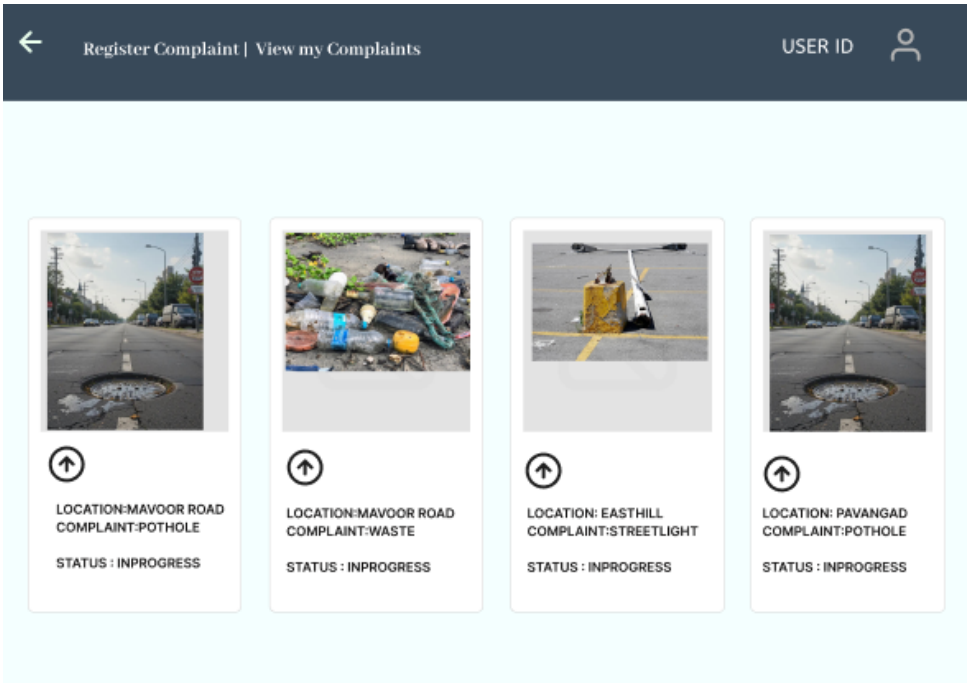

Figure 4.4: View Complaints Page

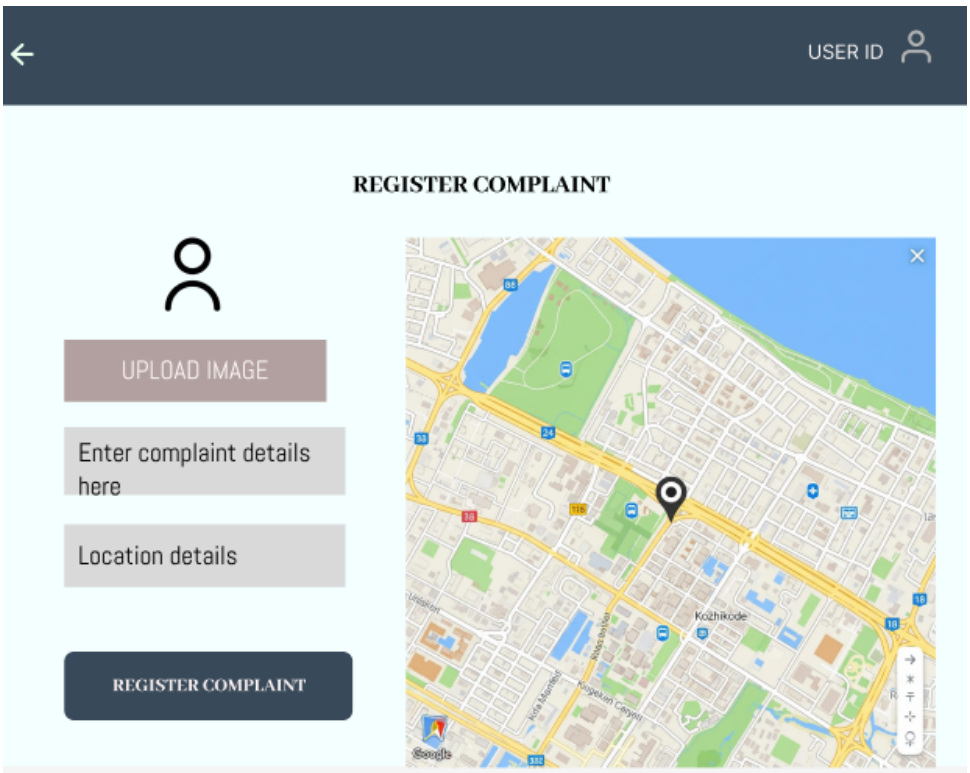

Figure 4.5: Complaint Registration Page

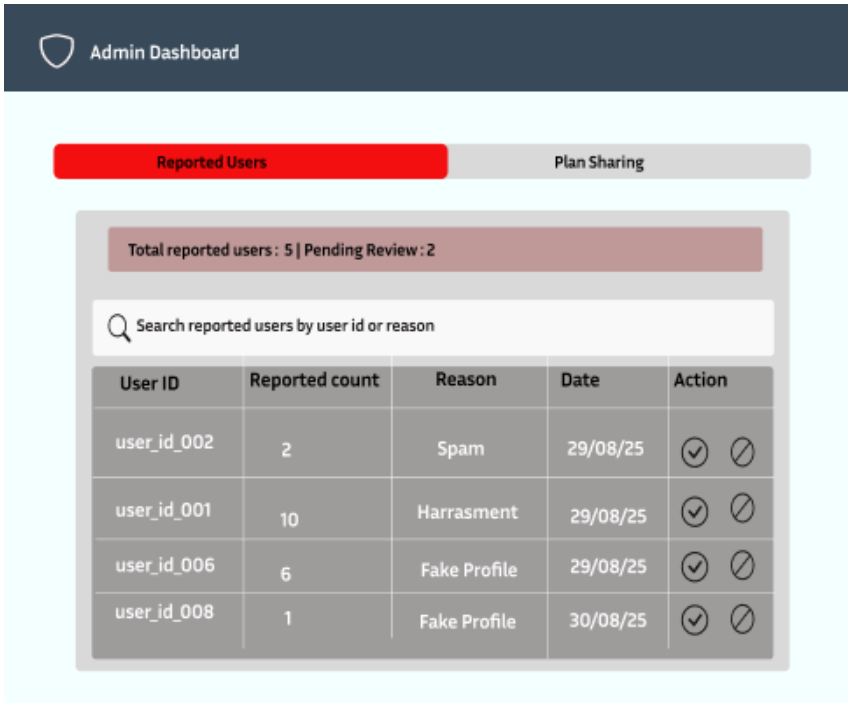

Figure 4.6: Admin Dashboard Page

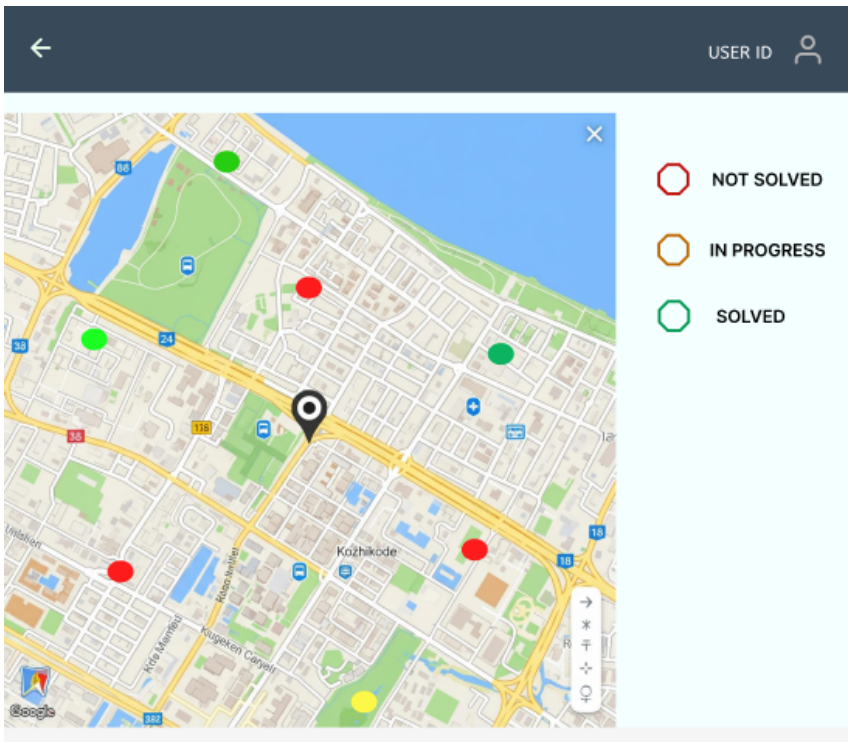

Figure 4.7: Map Visualization page

4.1.2 Data Flow Diagrams

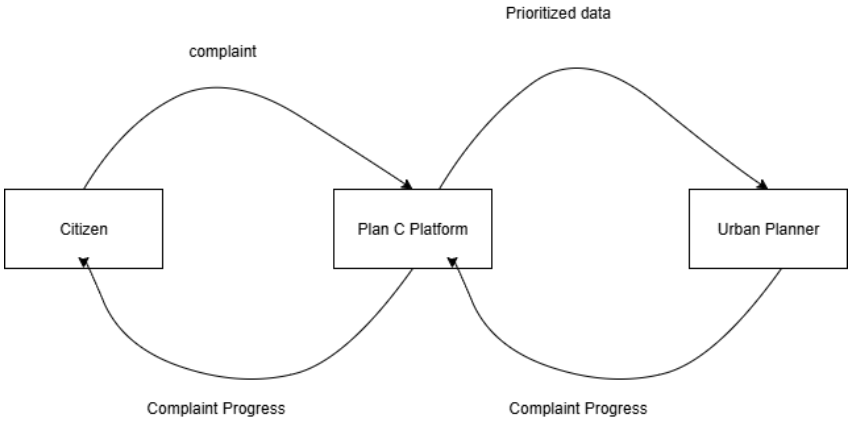

Figure 4.8: Data Flow Diagram Level 0

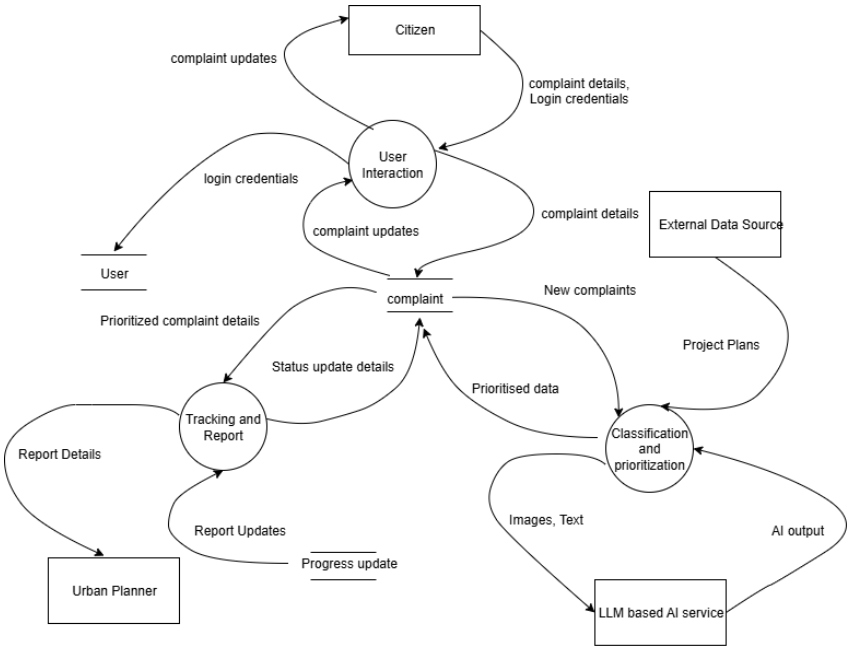

Figure 4.9: Data Flow Diagram Level 1

4.1.3 Use-Case Diagrams

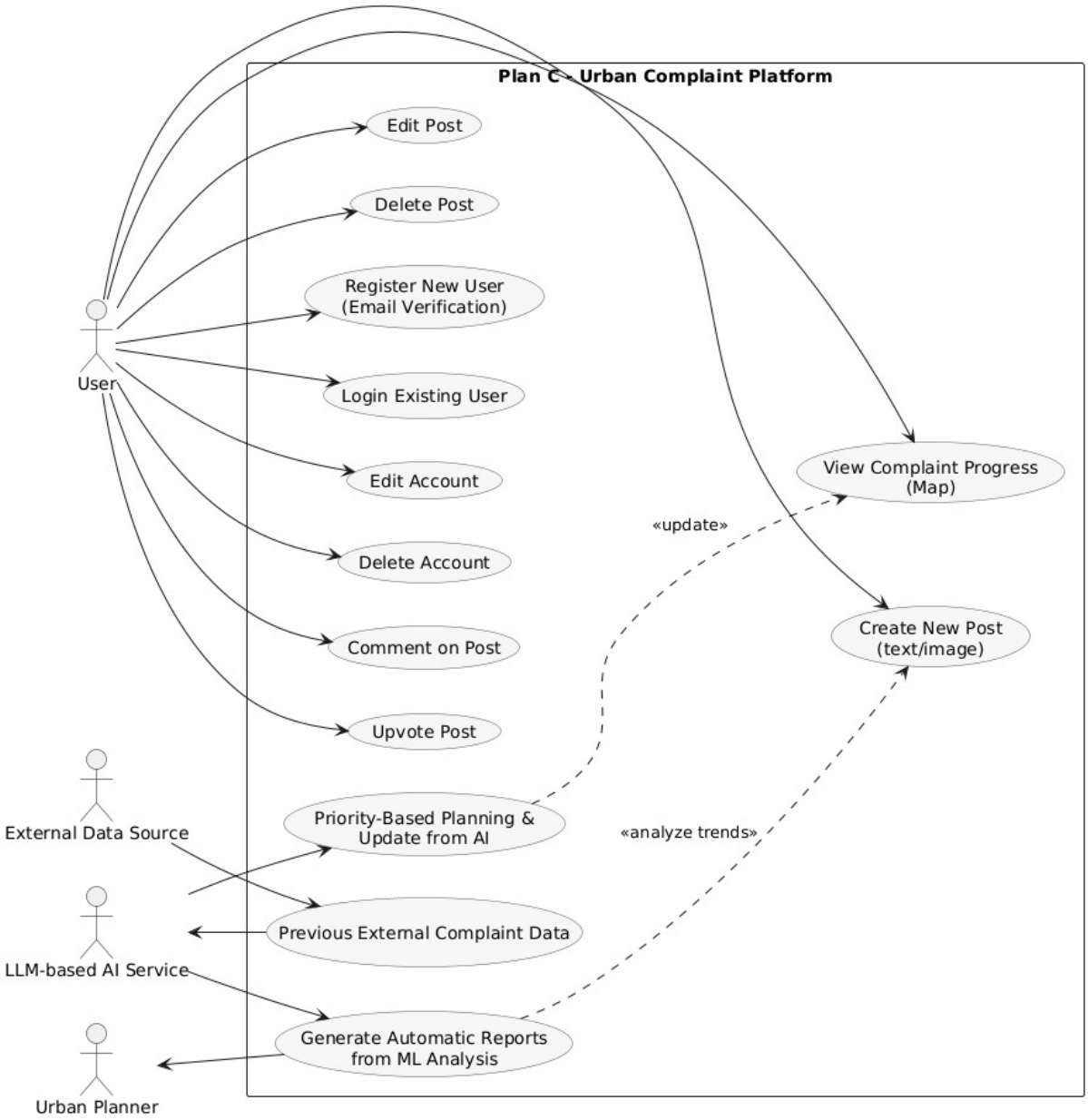

Figure 4.10: Usecase diagram

4.1.4 Database Design

The system uses two levels of storage:

- **Cloud Database:** Stores complaint data including text, images, and geolocation for

long-term persistence and scalability.

- **Local Database:** Maintains user credentials, votes, comments, and references to cloud-stored complaints for faster access and interaction.

Entity Descriptions

1. **User**

   Attributes: UserID, UserName, Email, Password, AccountCreationDate

   Description: Represents a person who can create posts, write comments, and file complaints.

   Relationships:

   - A user creates posts.
   - A user writes comments.
   - A user files complaints.

2. **Admin**

   Attributes: AdminID, Username, Password, Email

   Description: Represents a system administrator who manages complaints and maintains reports.

   Relationships:

   - An admin handles complaints filed by users.
   - An admin manages urban progress reports.

3. **Post**

   Attributes: PostID, UserID, Title, Description, Location, MapPointer, Status, CreateDate, UpdateDate, NoOfUpvotes, Image

   Description: Represents posts created by users to report or discuss issues.

   Relationships:

   - A post is created by a user.
   - A post receives comments.

- A post is linked with an urban progress report.

4. **Comment**

   Attributes: CommentID, PostID, UserID, Content, CreateDate, UpdateDate, NoOfUpvotes, Image

   Description: Represents user comments written under posts.

   Relationships:

   - A comment belongs to a post.
   - A comment is written by a user.

5. **User Misconduct Report**

   Attributes: ComplaintID, Date, UserID, TypeOfTarget, TypeOfComplaint, Details, TargetID

   Description: Represents complaints filed by users, managed by admins.

   Relationships:

   - A complaint is filed by a user.
   - A complaint is handled by an admin.

6. **UrbanProgressReport**

   Attributes: ReportID, PostID, Status, LastUpdated, SourceIdentifier

   Description: Provides progress updates related to posts/issues.

   Relationships:

   - An urban progress report is linked to a post.
   - An admin manages the urban progress report.

4.1.5 ER Diagram

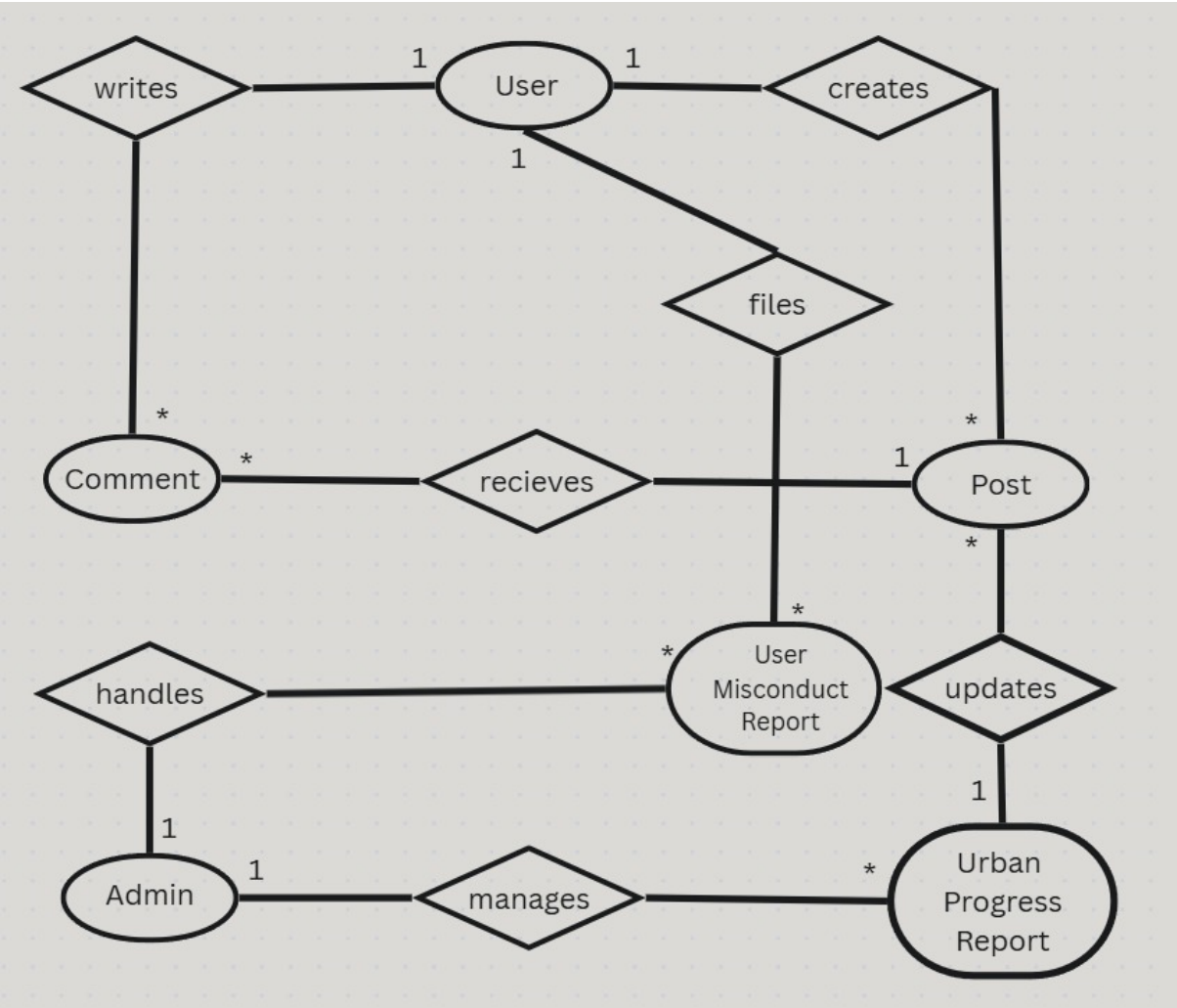

Figure 4.11: ER Diagram

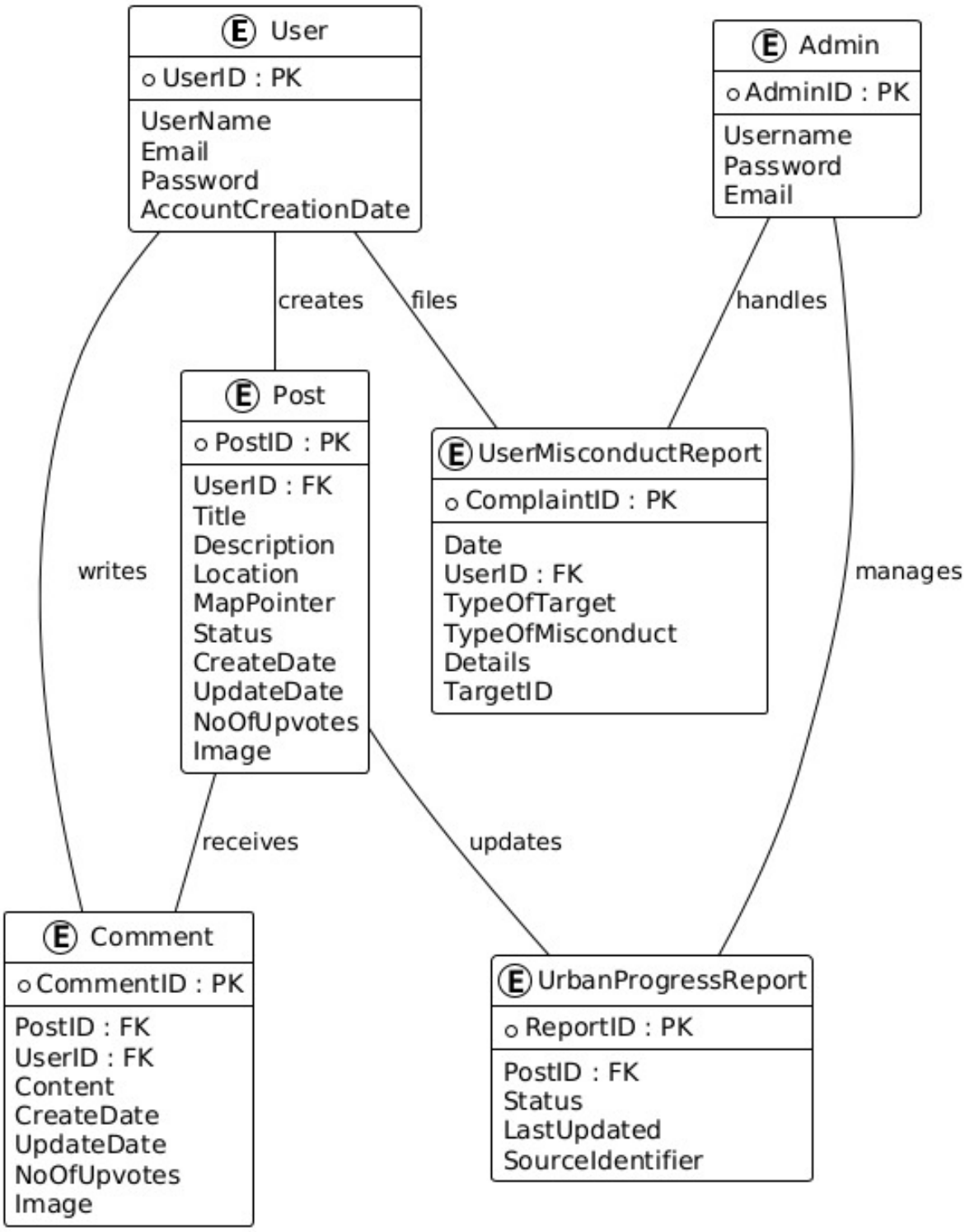

Figure 4.12: Class Diagram

4.1.6 Sequence Diagram

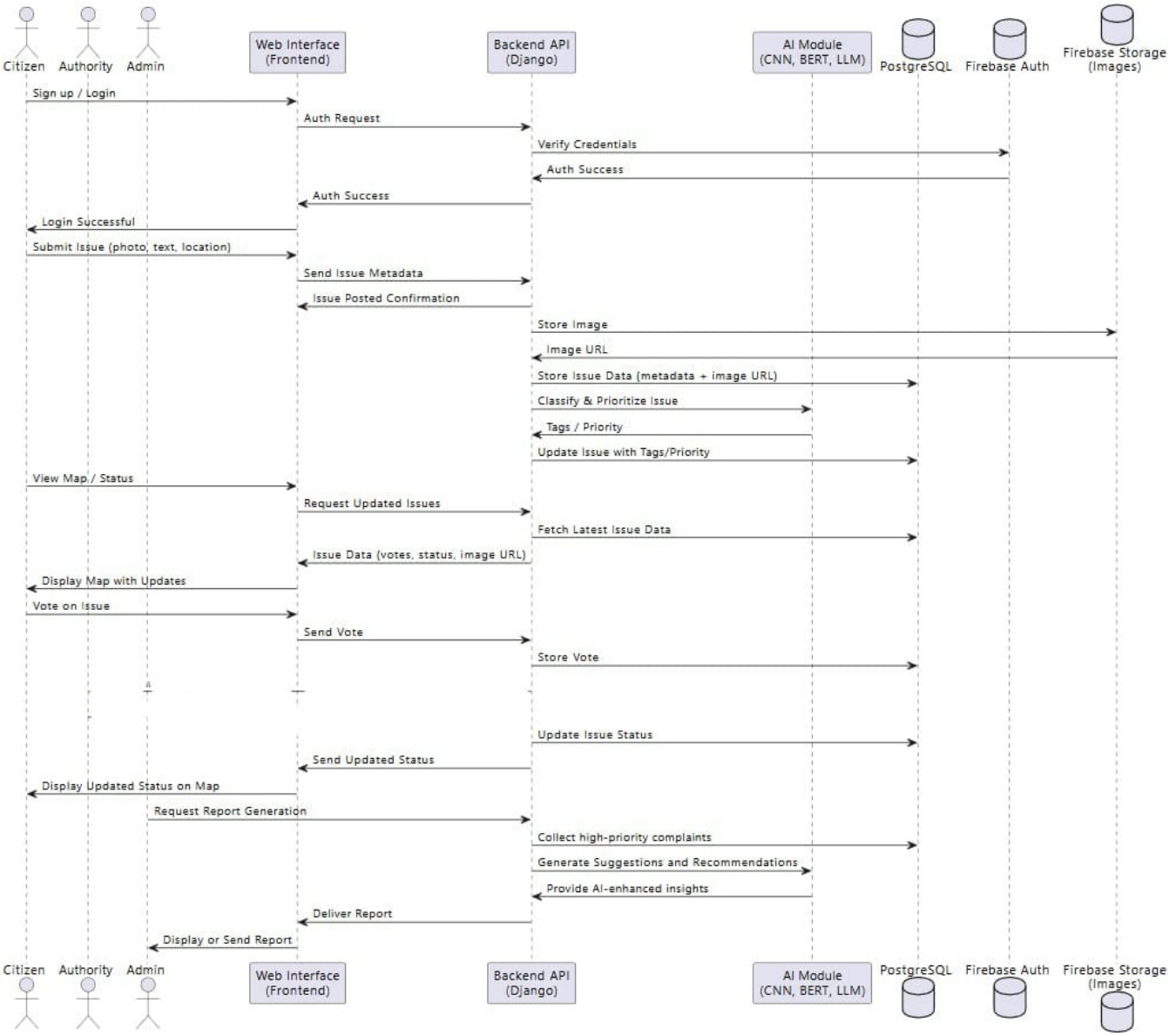

Figure 4.13: Sequence diagram

4.2 Implementation Steps

Module 1: User Authentication

1. Input: username, email, password from user.
2. Verify if the username exists in the database.
3. If user exists, validate password hash.
4. If credentials are correct:
   - Generate access token.
   - Return login success.

   Else:
   - Return login failure.
5. Output: Access token or error message.

Module 2: Complaint Submission

1. Input: description, image, location (map pin/address).
2. Upload image to cloud storage.
3. Save complaint details in cloud database: {userID, description, imageURL, location, timestamp}.
4. Store a reference of the complaint in the local database for faster access.
5. Output: Complaint ID (confirmation of successful storage).

Module 3: Complaint Browsing & Interaction

1. Input: Request for complaint feed.
2. Fetch complaints from cloud database along with local references.
3. Display posts in feed with {description, image, location, status}.
4. If user clicks Upvote:

  - Increment upvote count.
  - Store upvote in local database.
5. Update and refresh feed in real-time.
6. Output: Updated complaint feed with upvotes.

Module 4: AI-Powered Analysis

1. Input: Complaint text + image + upvotes.
2. Run CNN model on image and classify issue type (e.g., pothole, garbage).
3. Run BERT/NLP model on description and extract issue category and urgency terms.
4. Check issue criticality based on factors such as population density, topographical factors and number of people affected.
5. Fuse results with upvote weight:
   `Priority_Score = f(image_score, text_score, criticality_score, upvotes).`
6. Assign priority tag:
   - If score ≥ threshold_high → High Priority.
   - If threshold_low ≤ score < threshold_high → Medium Priority.
   - Else → Low Priority.
7. Optionally call LLM for reasoning and recommended action.
8. Store classification, tag, and AI-generated suggestions in complaint record.
9. Output: Complaint enriched with priority tag and AI insights.

Module 5: Report Generation

1. Trigger: Periodic or manual by admin.
2. Collect only the most voted and algorithmically high-priority complaints.
3. Include suggestions for possible solutions using LLM.

4. Generate report output in PDF/CSV or dashboard format.
5. Deliver reports to government officials via portal/email.
6. Output: Automated decision-support report of critical issues.

Module 6: Status Update

1. Input: Status change request from planner/official.
2. Options: Not Solved, In Progress, Solved.
3. Update Post table with: {status, lastUpdated, updatedBy, source}.
4. Reflect update in citizen-facing interface:
   - Change marker color on map.
   - Update complaint feed status.
   - Show map visualization of complaint status.
5. If AI detects external progress data:
   - Propose status update.
   - Validate and apply if approved by officials.
6. Output: Complaint with latest status visible to all users.

4.3 TESTING

Chapter 5

Results and Discussion

5.1 Results

5.2 Evaluation

Chapter 6

Conclusion and Future Works

References

[1] Design and Implementation of a Tactile Stimulation Device to Increase Auditory Discrimination, *IEEE Transactions on Haptics , (Volume: 10, Issue: 4, Oct.- Dec. 1 2017)*

[2] R. H. Gault, "Progress in experiments on tactual interpretation of oral speech", *The Journal of Abnormal Psychology and Social Psychology, vol. 19, pp. 155-159, 1924.*

[3] J. P. Das, "Stages of Reading Development," in Reading Difficulties and Dyslexia: An Interpretation for Teachers. *SAGE Publications India Pvt Ltd, 2009.*